

مسئله ۱: (a) با توجه به اینکه ورودی یک عدد حقیقی است و سه تانکس خروجی داریم بنابراین تابع دستبند باید از $\mathbb{R}$ به $\mathbb{R}^3$ باشد.

(b) با توجه به اینکه مجموع سه تانکس برابر یک می شود پذیرفته شده اند. البته شرط مرحله قبل را نیز دارند.

$$p(x) : \frac{x}{2} + \frac{x^2}{2} + 1 - \frac{x}{2} + \frac{x^2}{2} = 1$$

$$\tilde{p}(x) : 1 - 4x(1-x) - x(1-2x)^2 + 4x(1-x) + x(1-2x)^2 = 1$$

(c) مجموع تانکس ها برابر یک نمی شود.

$$q(x) : x + 1 - x + (x - \frac{1}{2})^2 \neq 1$$

$$x_1 = 0, y_1 = 1 \quad p(x) = (0, 0, 1) \Rightarrow \hat{y}_1 = 3 \quad \tilde{p}(x) = (1, 0, 0) \Rightarrow \hat{y}_1 = 1 \quad (d)$$

$$x_2 = \frac{1}{2}, y_2 = 2 \quad p(x) = (\frac{1}{4}, \frac{1}{8}, \frac{5}{8}) \Rightarrow \hat{y}_2 = 3 \quad \tilde{p}(x) = (0, 1, 0) \Rightarrow \hat{y}_2 = 2$$

$$x_3 = 1, y_3 = 3 \quad p(x) = (\frac{1}{2}, \frac{1}{2}, 0) \Rightarrow \hat{y}_3 = 1 \pm 2 \quad \tilde{p}(x) = (0, 0, 1) \Rightarrow \hat{y}_3 = 3$$

همان طور که مشاهده می شود عملکرد $\tilde{p}(x)$ بهتر از $p(x)$ است و توانسته مرتبه دیتا را به درستی دستبند کند.

$$X = \begin{bmatrix} 0.2 \\ 0.8 \end{bmatrix} \quad W' = \begin{bmatrix} 0.1 & 0.3 \\ 0.2 & 0.1 \end{bmatrix} \quad W'' = \begin{bmatrix} 0.2 \\ 0.2 \end{bmatrix} \quad b' = b'' = 0 \quad \text{sigmoid}(x) = \frac{1}{1+e^{-x}} \quad \text{مسئله ۲}$$

$$h' = W'^T X + b' = \begin{bmatrix} 0.1 & 0.2 \\ 0.3 & 0.1 \end{bmatrix} \begin{bmatrix} 0.2 \\ 0.8 \end{bmatrix} = \begin{bmatrix} 0.02 + 0.16 \\ 0.06 + 0.08 \end{bmatrix} = \begin{bmatrix} 0.18 \\ 0.14 \end{bmatrix}$$

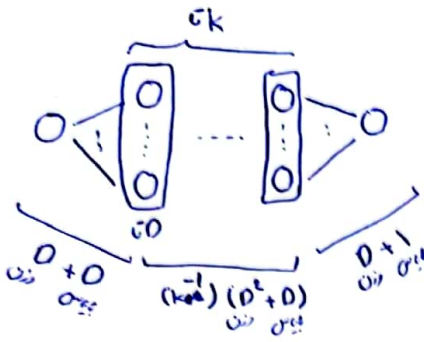
$$a' = \text{sigmoid}(h') = \begin{bmatrix} \text{sigmoid}(0.18) \\ \text{sigmoid}(0.14) \end{bmatrix} = \begin{bmatrix} 0.54 \\ 0.53 \end{bmatrix}$$

$$h'' = W''^T a' + b'' = \begin{bmatrix} 0.2 & 0.2 \end{bmatrix} \begin{bmatrix} 0.54 \\ 0.53 \end{bmatrix} = 0.214$$

$$a'' = \text{sigmoid}(h'') = [\text{sigmoid}(0.214)] = [0.55] \Rightarrow \hat{y} = 0.55, y = 0$$

$$L(W', W'') = -0 \times \log(0.55) - (1-0) \log(1-0.55) = 0.34$$

سوال ۳: (a)



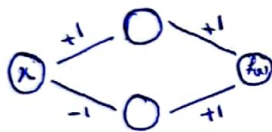
$$\begin{aligned} \text{len(parameters)} &= D+1 + (k-1)(D^2+D) + D+1 \\ &= (k-1)D^2 + (k+2)D + 1 \end{aligned}$$

شماره اول: $k=1$ $D=95 \Rightarrow \text{تعداد پارامتر} = 3D+1 = 3 \times 95 + 1 = 286$ $\text{تعداد نوتی} = (D+1)^k = 96$

(b)

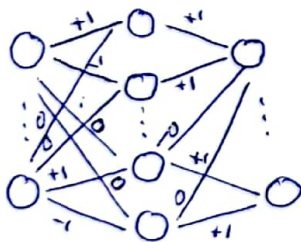
شماره دوم: $k=10$ $D=5 \Rightarrow \text{تعداد پارامتر} = (k-1)D^2 + (k+2)D + 1 = 9 \times 5^2 + 12 \times 5 + 1 = 286$ $\text{تعداد نوتی} = (6)^{10} = 60,466,176$

با توجه به بلای بودن تعداد پارامترها دقیق نمی‌دونم کدام سریع تر محاسبه می‌شود؛ در فیلتر کردن تعداد نوتی‌ها می‌توان گفت این شبکه سریع تر عمل می‌کند همچنین زمانی بیشتری را نسبت به شبکه اول تولید می‌کند که نشان دهنده توانایی آن در حل مسائل پیچیده‌تر را دارد.



$$f(x) = |x|$$

سوال ۴: (a) با در نظر گرفتن تابع فعال‌سازی ReLU با توجه به عدم است x همواره یکی از نوتی‌ها مقدار ۱ یا ۰ دارد که در نظر نتیجه خوبی مورد نظر مسئله است.



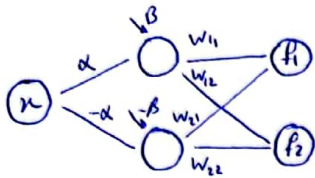
(b) کافی است برای هر نوتی در ردی x در نوتی در لایه میانی قرار دهیم و نوتی‌های اضافی را صفر در نظر بگیریم.

(c) با در نظر گرفتن این نکته که شبکه عصبی ANN می‌تواند به شکل مجموعه از خطوط در فیلتر کردن نتایج برای شبکه‌ای ساخت که دقیقاً تابع x^2 را شبیه سازی کند.

۱) برای هر لایه: $\alpha^l = \text{ReLU}(W^{lT} \alpha^{l-1})$
 ۲) لایه اول: $\alpha^1 = \text{ReLU}(W^{1T} X)$
 ۳) $\Rightarrow f(\lambda x) = |x| f(x)$ $\text{ReLU}(\lambda x) = |x| \text{ReLU}(x)$

نمونه‌ها درون داخل شبکه را می‌تواند به شکل خطی که در فیلتر مجموعه‌ای از آن شبکه را می‌سازد.

(d) خیر، نیاز است که ریوسته باشد.

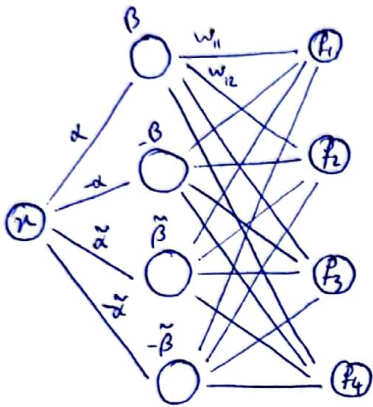


$$W = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$$

سوال ۵۹ (a)

$$\begin{cases} x_1 \in \Omega_1 \Rightarrow \alpha x + \beta < 0 \Rightarrow \begin{cases} f_1 = 0 + (-\alpha\beta) \\ f_2 = 0 + 0 \end{cases} \Rightarrow f_1 > f_2 \\ x \in \Omega_2 \Rightarrow \alpha x + \beta \geq 0 \Rightarrow \begin{cases} f_1 = 0 + 0 \\ f_2 = \alpha x + \beta + 0 \end{cases} \Rightarrow f_1 < f_2 \end{cases}$$

(b) با تقسیم حالت بالا برای کلاس با ضرایب زیر موازنه می‌شود.



$$W = \begin{bmatrix} 0 & 1 & 0 & 1 \\ 1 & 0 & 1 & 0 \\ 0 & 0 & 1 & 1 \\ 1 & 1 & 0 & 0 \end{bmatrix}$$

سوال ۸۶ (a)

$$\begin{aligned} z > 0, \alpha > 0 : \text{ReLU}(\alpha z) &= \alpha z = \alpha \text{ReLU}(z) \\ z < 0, \alpha > 0 : \text{ReLU}(\alpha z) &= 0 = \alpha \text{ReLU}(z) \end{aligned} \Rightarrow \text{ReLU}(\alpha \cdot z) = \alpha \text{ReLU}(z)$$

میان $\lambda_0 \lambda_1$ - دانج

$$\begin{aligned} \lambda_0 \lambda_1 \cdot \text{ReLU} \left[\frac{1}{\lambda_0 \lambda_1} \beta_1 + \lambda_1 \text{ReLU} \left(\frac{1}{\lambda_0} \beta_0 + \lambda_0 x \right) \right] &= \text{ReLU} \left[\frac{\lambda_0 \lambda_1}{\lambda_0 \lambda_1} \beta_1 + \lambda_0 \lambda_1 \cdot \text{ReLU} \left(\frac{1}{\lambda_0} \beta_0 + \lambda_0 x \right) \right] \\ &= \text{ReLU} \left[\beta_1 + \lambda_1 \lambda_0 \cdot \text{ReLU} (\beta_0 + \lambda_0 \lambda_0 x) \right] \end{aligned}$$

سوال ۷٪ (a) این مسئله یک مسئله دسته‌بندی باینری است بنابراین می‌توان با یک نورون استفاده از سیگنویر مسئله را حل کرد که عدد خروجی احتمال اسیم بودن ایمیل را نشان می‌دهد. هم می‌توان با استفاده از دو نورون و سخت‌کس اینکار را انجام داد.

(b) دیتاست MNIST شامل اعداد دستنویس بین ۰ تا ۹ است پس ۱۰ نورون خروجی نیاز داریم و باید از softmax استفاده کنیم.

(c) برای ریسینی قیمت خونه به یک نورون خروجی و یک تابع فعال ساز خطی نیاز داریم.

(d) تابع tanh یک عدد بین -۱ و ۱ تولید می‌کند و می‌تواند به هم فعال ساز یک مسئله دسته‌بندی باینری باشد که خروجی ۱۰۰ احتمال کلاس اول و ۰-۱۰۰ احتمال کلاس دوم را نشان دهد.

(e) با توجه به اینکه برای پیاده سازی سیگنویر به یک نورون نیاز است ما به softmax هم نیاز داریم اما خروجی softmax می‌تواند به عنوان یک احتمال نهایی که مجموع احتمال‌ها برابر یک است در نظر گرفته شود. همچنین softmax قابلیت ریسینی از چندین کلاس را دارد.

(f) تفاوت اصلی در بازه عدد خروجی است که تابع سیگنویر خروجی بین ۰ و ۱ می‌دهد اما تابع فعال ساز خطی این محدودیت را ندارد و به صورت کلی فقط در جایی می‌توان از سیگنویر استفاده کرد که خروجی بین ۰ و ۱۰۰ معنادار باشد مثلاً و راندی بین ۰ و ۱۰۰ را ریسینی کنیم مثل میزان تمایل در پوشش جلی بپوشیدن.

(g) در صورتی که خواهیم دسته‌بندی چند کلاس داشته باشیم استفاده از softmax که مدل را مجبور به اولویت‌دادن به یک کلاس می‌کند منطقی نیست می‌توانیم از توابع سیگنویر یا tanh به عنوان تابع فعال ساز استفاده کنیم که هر نورون احتمال تعلق داشتن داده به همان کلاس را نشان می‌دهد و دیگر مجموع نورون‌ها برابر ۱ نخواهد بود.

سوال ۸٪ (a) چند وجهی محذب از ترکیب چند نیم فضا تشکیل شده است بنابراین داریم: $P = \bigcap_{i=1}^n \{x | Ax_i + b_i \leq 0\} = \{x | Ax + b \leq 0\}$ در این حالت می‌توانیم یک شبکه عصبی با دو لایه تعریف کرد که در لایه اول هر نورون نشان دهنده یک نیم فضا باشد و لایه دوم عملکرد AND برای لایه اول اجرا کند. تقریباً شبیه سوال یک که حل شد.

(b) در صورت داشتن چند وجهی غیر محذب ابتدا باید نواحی محذب را شناسایی کرد سپس با استفاده از OR به ناه نیم محذب مورد نظر رسید که برای این منظور به سه لایه نیاز داریم که در لایه اول نواحی محذب را شناسایی می‌کند و در لایه آخر عملکرد OR را پیاده‌سازی می‌کند.

این سوال به از این تست می‌آید.